Statistical Inference and Modeling

2102575

Suwichaya Suwanwimolkul[1]

[1]Electrical Engineering
Chulalongkorn University

Semester II, 2025

Lectures

- Lecture 1: Introduction
- Lecture 2: Model selection
- Lecture 3: Resampling
- Lecture 4: Linear regression
- Lecture 5: Robust regression
- Lecture 6: Classification
- Lecture 7: Logistic regression
- Lecture 8: Neural networks
- Lecture 9: K-nearest neighbor
- Lecture 10: Bayesian decision, LDA, QDA, and naive Bayes
- Lecture 11: Support vector machine
- Lecture 12: Tree-based methods
- Lecture 13: Bagging, random forecast, boosting
- Lecture 14: Unsupervised learning and clustering algorithms
- Lecture 15: Unsupervised learning and PCA
- Lecture 16: Gaussian mixture model clustering
- Lecture 17: DBSCAN clustering

7. Classification

2102575

Suwichaya Suwanwimolkul[1]

[1]Electrical Engineering
Chulalongkorn University

Semester II, 2025

What is a classification?

Classification is a process that assigns the observation to a class

- Often, a method first predicts the probability of a class of a qualitative variable; or it provides rules to assign observations to a class;
- A classification technique is called **classifier**.

Example

A classification of patients heart disease condition: mild, moderate, and severe (3 classes) using $\boldsymbol{x} := [\text{LDL, BMI, alcohol, age}]$ as predictors.

1 sample → 2 Class → Binary (จบตรงนี้)
3+ Class → Multi-classes
เป็นได้หลาย Class พร้อมกัน → Multi-Label

A classifier can provide threshold-based rules on x:

- $\hat{y} =$ mild, if LDL < 40 and BMI < 30.
- $\hat{y} =$ moderate, if (LDL $\in [100, 120]$ and alcohol > 10) or (LDL $\in [120, 200]$ and age > 40) or (BMI $\cdot$ alcohol $\in [1, 100]$).
- $\hat{y} =$ severe, uf (LDL > 200) or (BMI $\cdot$ alcohol > 200).

A classifier approximate the probability of each class (given an input features $\boldsymbol{x}$) via mathematical functions. A classifier maps the input $\boldsymbol{x}$ to a probabilistic value (0-1), $\hat{f}(\cdot) : \mathcal{R}^n \Rightarrow [0, 1]^C$:

$$p(Y = c|\boldsymbol{x}) = \hat{f}(y_c|\boldsymbol{x}, \theta)$$

← We can use ML to estimate the class by predicting the probability.

where θ is the model parameter needed to be identified.

Settings.

- **Training process:** involves how to find good thresholds or the best parameter θ, using training data.
- **Testing process:** given a new value of $\boldsymbol{x}$, we predict $\hat{y}$ using the estimated parameter θ.

Why not linear regression?

Suppose we try to predict a medical condition of three cases: stroke, drug overdose, epileptic seizure.

$$\text{Choice 1:} \quad \hat{y} = \begin{cases} 0, & \text{normal} \\ 1, & \text{stroke} \\ 2, & \text{drug overdose} \\ 3, & \text{epileptic seizure} \end{cases} \qquad \text{Choice 2:} \quad \hat{y} = \begin{cases} 0, & \text{normal} \\ 1, & \text{epileptic seizure} \\ 2, & \text{stroke} \\ 3, & \text{drug overdose} \end{cases}$$

- If you use a linear regression, *i.e.*, $\hat{y} = \boldsymbol{x}_{\text{Test}}^T \boldsymbol{\beta}$, then the value of $\hat{y}$ could be thresholded into 4 categories, just like in Choice 1 and 2.
- However, the value of $\hat{y} = \boldsymbol{x}_{\text{Test}}^T \boldsymbol{\beta}$ cannot catch up with the lack of natural ordering in these labels.
- The different distance between predictions, *e.g.*, 0 vs 1 or 0 vs 2 should be the same. Thus, the **first reason** is the mapping value in linear regression cannot reflect the unnatural ordering of the labels.

Why not linear regression?

If there are only two possibilities: stroke or drug overdose.

$$\text{Choice 1: } y = \begin{cases} 0, & \text{stroke} \\ 1, & \text{drug overdose} \end{cases} \qquad \text{Choice 2: } \tilde{y} = \begin{cases} 0, & \text{drug overdose} \\ 1, & \text{stroke} \end{cases}$$

- We can fit a linear regression using the first choice of encoding and predict drug overdose if $y > 0.5$ (or for choice 2, predict stroke if $\tilde{y} > 0.5$).
- However, if we use linear regression, some of our estimates might be outside the [0, 1] interval, making them hard to interpret as probabilities!
- **Second reason:** as we know in linear regression $\hat{y} = \boldsymbol{x}_{\text{Test}}^T \boldsymbol{\beta}$, some of our estimates might be outside [0, 1]. So, we need a better mapping function to ensure that $\hat{y}$ can return the predicting probability.

Discriminative vs. generative approach for classifiers

β = unknown

Discriminative approach: ↗ การ learn likelihood

- Learn the parameters from $p(Y = c|\boldsymbol{X})$, directly.

$p(y|\beta)$: likelihood

$p(\beta|y)$: posterior

MLE ที่แหล่ะ ⇝ $\hat{\boldsymbol{\beta}} = \underset{\boldsymbol{\beta}}{\textbf{argmax}}\ p(Y = c|\boldsymbol{X}, \boldsymbol{\beta})$

- Learn mappings from inputs to classes (least-squares, decision trees).
- *e.g.*logistic regression, KNN, NN classifier, SVM, decision trees., logistic regression.

Generative approach:

- Learn the parameters from $p(\boldsymbol{X}|Y)$ or $p(\boldsymbol{X}, Y)$ or $p(\boldsymbol{X})$

$$\boldsymbol{\beta}^* = \underset{\boldsymbol{\beta}}{\textbf{argmax}}\ \frac{\log p(\boldsymbol{X},\ \boldsymbol{\beta}, Y = c)}{p(Y=c)} = \log \frac{p(\boldsymbol{X}, \boldsymbol{\beta}|Y = c)}{\cancel{p(Y=c)}}$$

- We then use Bayes' theorem to flip these around into estimates for the class from $p(Y = c|\boldsymbol{X})$.

ความน่าจะเป็นที่ y อยู่ Class $c \in C$ ภายใต้ X; β^* เท่าไหร่?

$$c^* = \underset{c}{\textbf{argmax}}\ \log p(Y = c|\ \boldsymbol{X};\ \boldsymbol{\beta}^*) \propto \log p(Y = c, \boldsymbol{X};\ \boldsymbol{\beta}^*)$$

- Eg. LDA, QDA, Naive Bayes.

Binary classification performance

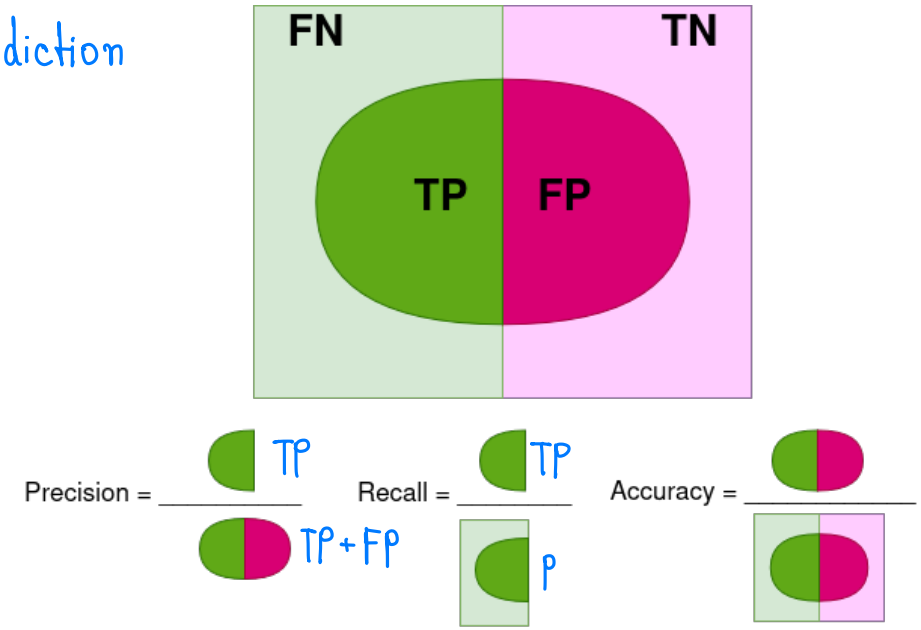
Confusion Matrix สำหรับ Binary Classification.

Suppose a response variable (y) has two outcomes: positive or negative.

Actual samples / Ground truth		Predicted Positive	Predicted Negative
		Predicted samples	
Actual Positive (P)		True Positive (TP)	False Negative (FN) $FN = P - TP$
Actual Negative (N)		False Positive (FP)	True Negative (TN) $TN = N - FP$

Prediction

Actual



- True positive (TP): correctly identified positive
- True negative (TN): correctly identified negative
- False positive (FP): incorrectly identified positive → type I
- False negative (FN): incorrectly identified negative → type II

Binary classification performance

$$\text{Recall} = \frac{TP}{P}$$ (ตาม ground truth)

สมมติว่าเรารู้ว่าคนป่วยเป็น Covid จริงๆ คือ 5 คน ใน 10 คน

↓ 5 → P

Recall คือใน 5 คนที่เป็น Covid จริงๆ นี้เรา predict ถูกว่าเป็นเท่าไหร่ ?

↓

TP

$$\text{Precision} = \frac{TP}{TP + FP} \rightarrow \frac{TP}{\hat{P}}$$ (ตาม classifier)

สมมติ Classifier เราบอกว่ามีคนป่วย 8 คน ($\hat{P}$)

และไม่ป่วย 2 คน จากคน 10 คน

↓

แต่จริงๆ เรามีคนป่วย (P) แค่ 5 คนเองนะ...

Standard classification indices

		Predicted Positive	Predicted Negative		
Actual samples / Ground truth	Actual Positive (P)	**True Positive (TP)**	**False Negative (FN)** $FN = P - TP$	**Recall**, Sensitivity, True Positive Rate (TPR) $\frac{TP}{P}$	**Type II error**, False Negative Rate (FNR) $\frac{FN}{P} = 1 - TPR$
	Actual Negative (N)	**False Positive (FP)**	**True Negative (TN)** $TN = N - FP$	**Type I error**, False Positive Rate (FPR) $\frac{FP}{N}$	**Specificity**, Selectivity, True Negative Rate (TNR) $\frac{TN}{N} = 1 - FPR$
		Precision $\frac{TP}{TP + FP}$			

Accuracy *

$$\frac{TP + TN}{P + N}$$

F1 → harmonic mean of precision and recall.

$$\frac{2Precision \cdot Recall}{Precision + Recall} = \frac{2TP}{2TP + FP + FN}$$

Standard classification indices

Sensitivity or recall: $$\text{TPR} = \frac{\text{correctly predicted positive}}{\text{total positive}} = \frac{\text{TP}}{\text{TP}+\text{FN}}$$

$$\text{FPR} = \frac{\text{incorrectly predicted positive}}{\text{total negative}} = \frac{\text{FP}}{\text{FP}+\text{TN}}$$

Specificity: $$\text{TNR} = \frac{\text{correctly predicted negative}}{\text{total negative}} = \frac{\text{TN}}{\text{TN}+\text{FP}}$$

$$\text{FNR} = \frac{\text{incorrectly predicted negative}}{\text{total positive}} = \frac{\text{FN}}{\text{TP}+\text{FN}}$$

Accuracy: $$\text{ACC} = \frac{\text{all correctly prediction}}{\text{total population}} = \frac{\text{TP}+\text{TN}}{\text{P}+\text{N}}$$

- By adjusting a classifier's hyperperameters, if TPR increases, so does FPR.
- in medical applications when positive is rare, it's more challenging to obtain a low FPR (aka false alarm) or high specificity

More classification performance indices

When observations contain an imbalance between two classes:

$$\text{Prevalence:} \quad = \frac{\text{total positive}}{\text{total population}} = \frac{\text{P}}{\text{P+N}}$$

$$\text{False discovery rate (FDR, FPR)} \quad = \frac{\text{incorrectly predicted positive}}{\text{predicted positive}} = \frac{\text{FP}}{\text{FP+TP}}$$

$$\text{Positive predictive value (PPV):} \quad = \frac{\text{correctly predicted positive}}{\text{predicted positive}} = \frac{\text{TP}}{\text{TP+FP}}$$

$$= 1 - \text{FDR (or precision)}$$

$$\text{F1 score} \quad = \text{harmonic mean of precision and sensitivity}$$

$$= 2 \times \frac{\text{PPV} \cdot \text{TPR}}{\text{PPV+TPR}} = \frac{2\text{TP}}{2\text{TP+FP+FN}}$$

F1 score finds the (harmonic) mean of two rates and the precision rate takes into account the portion of positive in data.

Trade-off in classification

Sensitivity-specificity trade-off in medical applications.

- A classifier is associated with a threshold or some hyper-parameters which control the fraction of TP and FP.
- If a classifier is aimed to detect a disease condition more correctly, it has a high sensitivity (sensitive to detect such disease).
- However, when it detects more positives, out of those detected positives may be incorrect; it pays a price for FP and a drop in specificity (the prediction is not specified enough to distinguish between the actual and false positives).

Trade-off in classification

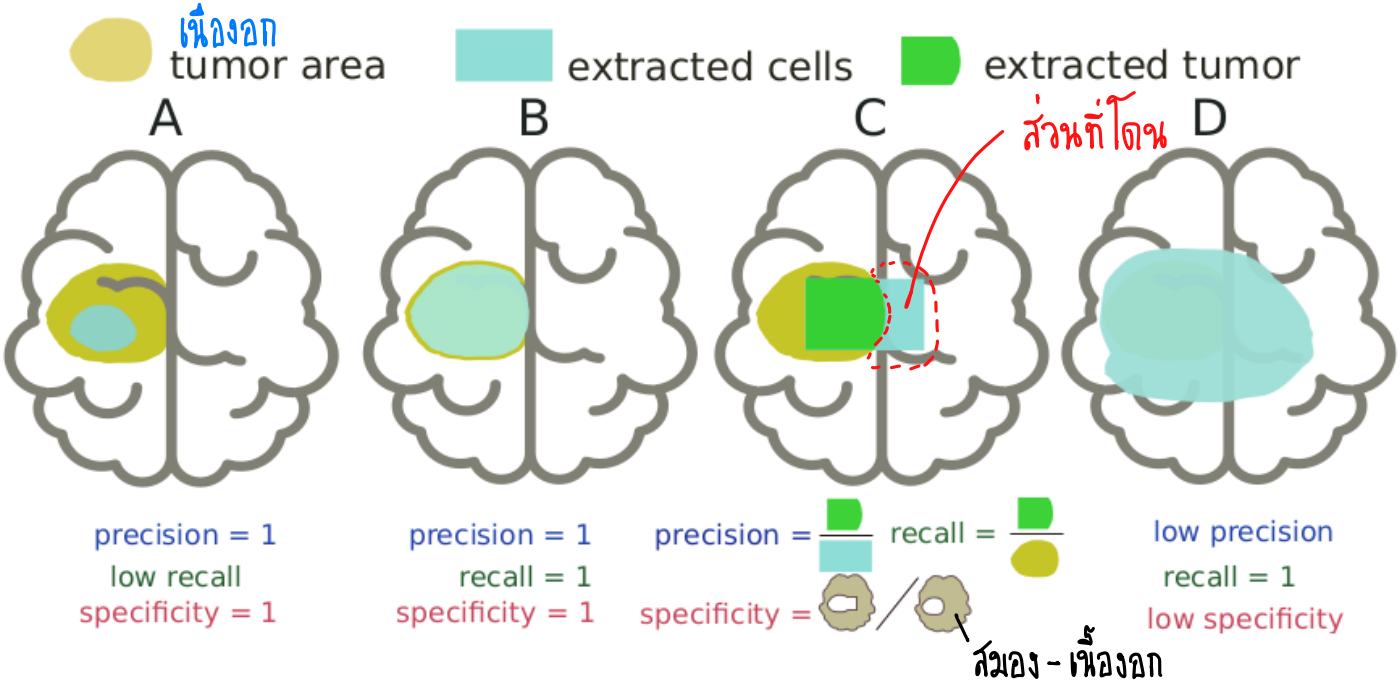
Precision-recall trade-off in information retrieval.

- A user creates a search query (from a universe of data items) and a relevant list of items is retrieved for the user.
- The query has high precision if a large fraction of the retrieved results are relevant.
- The query has high recall if it retrieves a large fraction of all relevant items in the universe – this application focuses less on TNR because it is typically high.

Trade-off in detecting brain tumor

Specifity $\frac{TN}{N}$ | Recall $\frac{TP}{P}$ | Precision $\frac{TP}{\hat{P}}$

A neuro surgeon makes a decision to remove a brain tumor



- A conservative move (A): avoid to remove healthy cells by extracting only little.
- A bold move (D): all tumor must be gone, but unavoidably remove healthy cells.
- Always consider a combination of (sens,spec) or (precision,recall) – there is a price to pay; when one index increases, the other index would drop.

Example: LDA performance on default data

Seting: 10,000 training samples, 3.33% of training samples defaulted

		Predicted default: Yes	Predicted default: No	Total	
Actual default	Yes	81 TP	252 TN	333	P
	No	23 FN	9,644 FP	9,667	N
	Total	104	9,896	10,000	
		$\hat{P}$	$\hat{N}$		

Performance of LDA on training data:

$\text{FPR} = 23/9{,}667 = 0.238\ \%$

$\text{FNR} = 252/333 = 75.5\ \%$

$\text{ACC} = (81 + 9{,}644)/10{,}000 = 97.25\%$

$\text{F1} = 37.07\%$

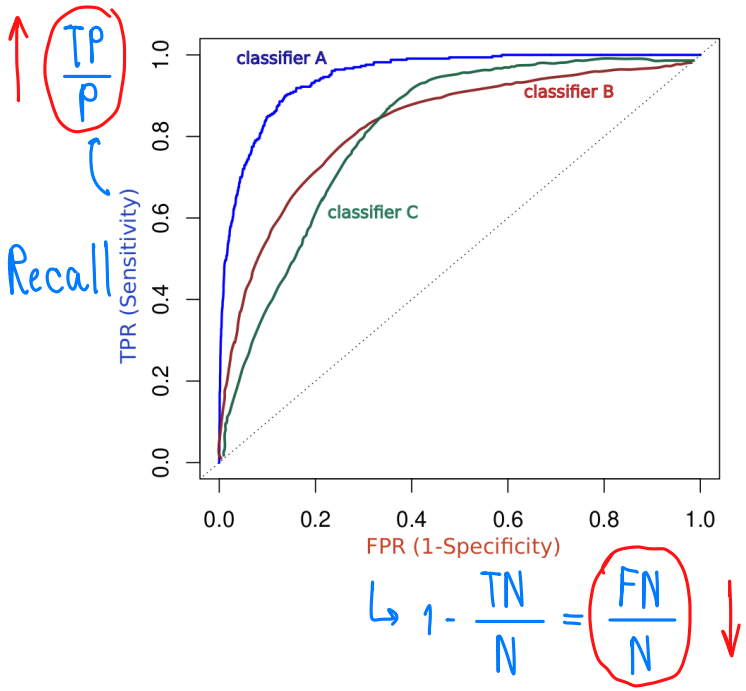
$$F1 = 2\left(\frac{1}{\text{Recall}} + \frac{1}{\text{Precision}}\right)$$

Notes:

- Type I error (FP): incorrectly predicted defaults, type II error (FN): incorrectly predicted non-defaults – a credit card company may wish to avoid FN (more serious) while FP is probably less problematic.
- In this example, LDA has a low sensitivity of 24.3% and specificity of 99.76%.
- Overall accuracy is high while a low sensitivity can tell us the type of error that is more concerned: LDA missed a lot of true defaulters

Receiver operating characteristic (ROC)

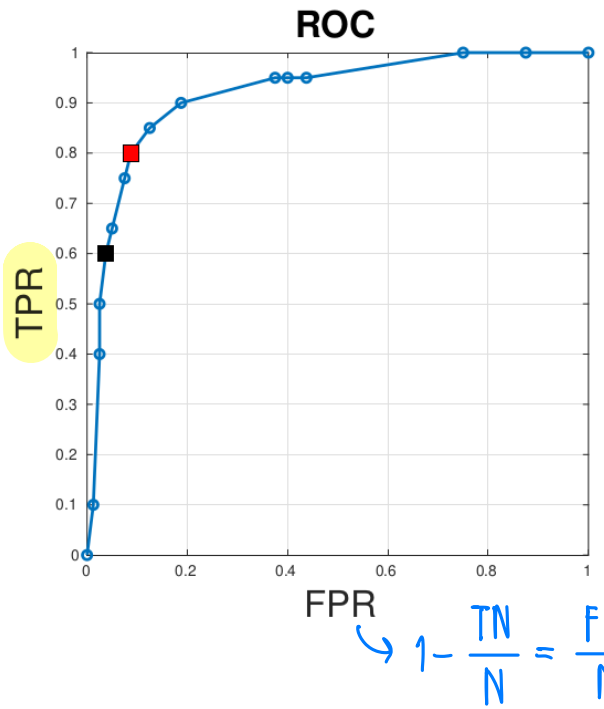
Receiver operating characteristic (ROC) is a plot between FPR (sensitivity/recall) and TPR (specificity).
Example: assign Y to the class '1' if $P(Y = 1|X = x) >$ threshold.



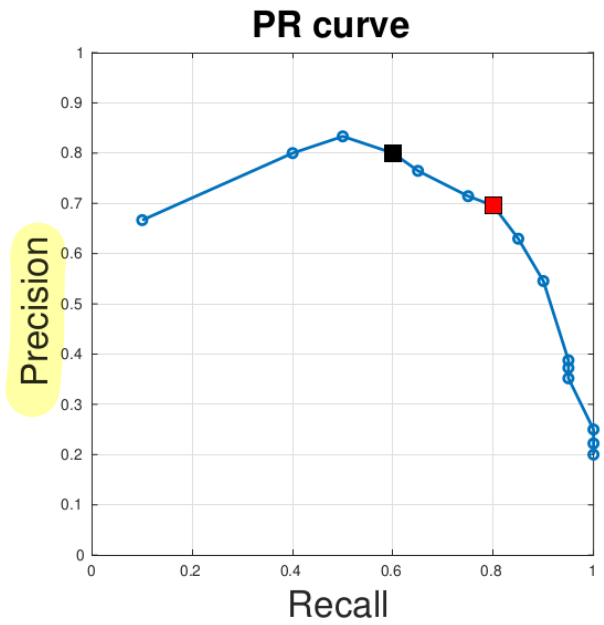
- Each point on ROC is associated with a **threshold/hyperparameter** of a classifier.
- A classifier performs better than a random guess, if ROC lies above the diagonal line.
- A better classifier has the ROC curve toward **the top left corner**.
- Overall performance is summarized over all possible thresholds is given by **area under curve (AUC)**.

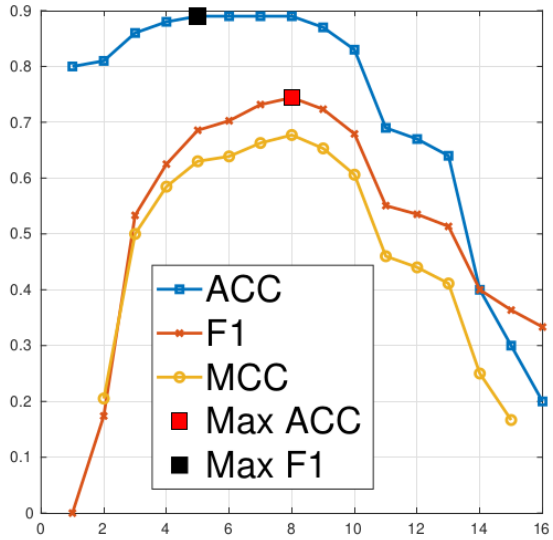
① เราจะเลือกบริเวณที่ Recall สูง, FPR ต่ำ หรือ
② ใครให้พื้นที่มากกว่ากัน ก็ชนะ

Precision-recall curve



$$\text{FPR} \hookrightarrow 1 - \frac{TN}{N} = \frac{FN}{N}$$





- **Black square** is the point for which ACC is maximum.
- **Red** is the point for which F1 is maximum.
- **Both black and red** points lie on the top-left corner of ROC, and on the top-right corner of PR (Precision-Recall) curve, indicating that these points are efficient.
- In practice, an operating point can be chosen by selecting the classifier's hyperparameter that maximizes some index (F1, MCC, ACC) on validated data.

Multi-class classification

Let's consider a C-class classification labeled as G1, G2, G3, G4, G5 ($C = 5$). The confusion matrix $\in \mathbb{R}^{C \times C}$ contains the number of samples in each predicted class is calculated according to the binary classification indices by picking a class of interest positive, and regard the remaining classes as negative.

Actual \ Predicted	G1	G2	G3	G4	G5	sum
G1	**9**	2	2	4	2	19
G2	1	**10**	2	2	5	20
G3	2	5	**14**	1	5	27
G4	1	5	2	**8**	3	19
G5	1	2	2	2	**8**	15
sum	14	24	22	17	23	**100**

Evaluation metrics to consider:

- Accuracy
- Precision, recall, F1 in *per-class performance*
- Precision, recall, F1 using *macro-averaging*
- ... ... *micro-averaging*
- ... ... *weighted-averaging*

Accuracy

A C-class classification labeled as G1, G2, G3, G4, G5 ($C = 5$).

$$\text{Accuracy} = \frac{\sum_c TP_c}{\text{all predictions}}$$

ผลรวมในแนวทะแยง

For the confusion matrix below, the accuracy is $\frac{9+10+14+8+8}{100} = 0.49$

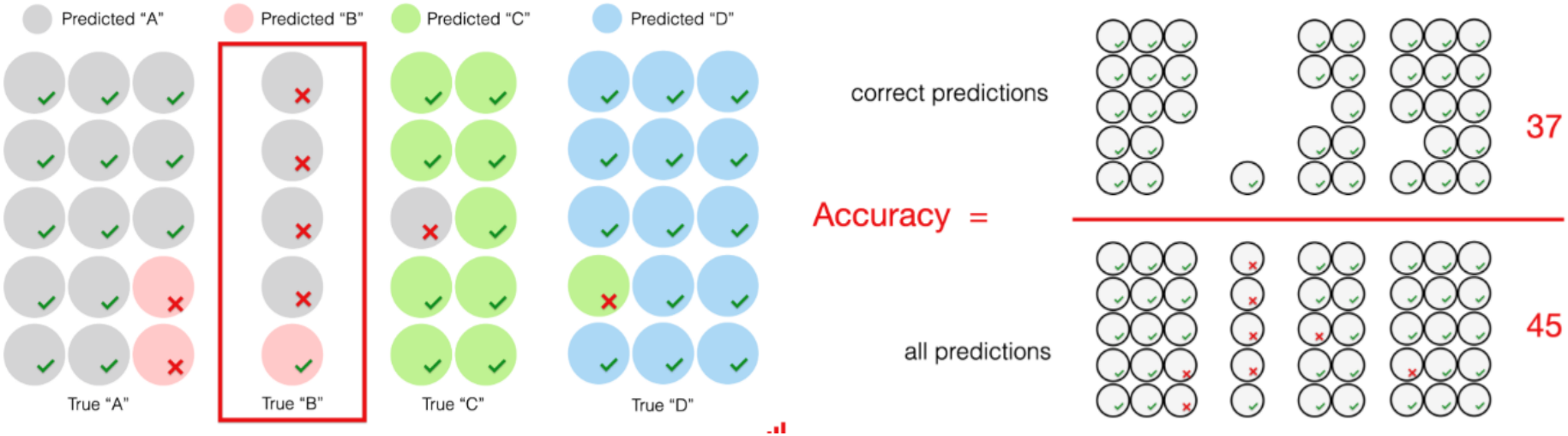
Actual \ Predicted	G1	G2	G3	G4	G5	sum
G1	**9**	2	2	4	2	19
G2	1	**10**	2	2	5	20
G3	2	5	**14**	1	5	27
G4	1	5	2	**8**	3	19
G5	1	2	2	2	**8**	15
sum	14	24	22	17	23	100

The good, the bad of accuracy for multi-classes.

- The good. Straightforward!
- The bad. Disregard class balance and the cost of different errors.

What is class balance and the cost of different errors?

Problem with accuracy



Let's look at an example from [1], the model did not do a very good job of predicting Class "B." However, since there were only 5 instances of this class, it did not impact the the overall accuracy is $37/45 = 82\%$!

Performance per class: confusion matrix of C-class

Performance per class.

$$\text{Precision}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c} \quad \text{and} \quad \text{Recall}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c}$$

Then,

$$\text{F1}_c = 2\frac{\text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}.$$

From the confusion matrix example, we can calculate precision, recall, and F1 of class G3 as follows:

$$\text{Precision}_{\text{G3}} = \frac{14}{22} = 63.6\% \quad \text{Recall}_{\text{G3}} = \frac{14}{27} = 51.9\% \quad \text{F1}_{\text{G3}} = 2\frac{\text{Precision}\cdot\text{Recall}}{\text{Precision}+\text{Recall}} = 57.1\%$$

Actual \ Predicted	G1	G2	G3	G4	G5
G1	9	2	2	4	2
G2	1	10	2	2	5
G3	2	5	14	1	5
G4	1	5	2	8	3
G5	1	2	2	2	8

22

G3 = positive	FP	FN	TP	TN

The good/the bad with per-class performance.

- The good...
  - Specific to classes of interest.
  - Helpful when dealing with imbalanced classes. Clearly expose issues in each class.
- The bad...The more classes, the more metrics you get.

Micro, Macro, and weighted-averaging

Three common ways summarize Precision and Recall across different classes as one value:

1. **Macro-averaging**
2. **Micro-averaging**
3. **Weighted-averaging**

Micro vs. Macro-averaging

1 **Macro-averaging** computes each precision_c and then average.

$$\text{Precision} = \frac{1}{C}\sum_{c=1}^{C}\frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c}, \quad \text{Recall} = \frac{1}{C}\sum_{c=1}^{C}\frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c}, \quad \text{and} \quad \text{F1} = 2\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

In fact, the above formulation can be re-written as ...

$$\text{Precision} = \frac{1}{C}\sum_{c=1}^{C}\text{Precision}_c \quad \text{and} \quad \text{Recall} = \frac{1}{C}\sum_{c=1}^{C}\text{Recall}_c.$$

- *Good ...*

  Useful when all classes are equally important, and you want to know how well the classifier performs on average across them.

  Useful when you have an imbalanced dataset and want to **ensure each class equally contributes** to the final evaluation.

- *Bad...* It can still distort the perception of performance; often this depends on the number of classes (C).

  Small C. It can make the classifier look "worse" due to low performance in an unimportant class.

  Large C. It can disguise poor performance in the critical minority class when the overall number of classes is large. In this case, **the "contribution" of each class is diluted**.

Micro vs. Macro-averaging

2 **Micro-averaging** takes a sum of all TP_c's and then compute precision/recall.

$$\text{Precision} = \frac{\sum_c \text{TP}_c}{\sum_c (\text{TP}_c + \text{FP}_c)}, \quad \text{Recall} = \frac{\sum_c \text{TP}_c}{\sum_c (\text{TP}_c + \text{FN}_c)}, \quad \text{and} \quad \text{F1} = \text{Precision} = \text{Recall}$$

Likewise, the above formulation can be re-written as ...

$$\text{Precision} = \frac{\text{TP}_1 + \text{TP}_2 + ... + \text{TP}_C}{\text{TP}_1 + \text{FP}_1 + \text{TP}_2 + \text{FP}_2 + ... + \text{TP}_C + \text{FP}_C}$$

$$\text{Recall} = \frac{\text{TP}_1 + \text{TP}_2 + ... + \text{TP}_C}{\text{TP}_1 + \text{FN}_1 + \text{TP}_2 + \text{FN}_2 + ... + \text{TP}_C + \text{FN}_C}$$

Note. Every FP for one class is an FN for another class. Thus, micro-average precision and recall results in the same number.

- *Good ...*

  Actually, Precision = Recall = Accuracy. Thus, it's advantages are similar to the accuracy's. Good when you want to account for the total number of misclassifications in the dataset. It gives equal weight to each instance and will have a higher score when the overall number of errors is low.

- *Bad...*

  Micro-averaging can overemphasize the performance of the majority class, especially when it dominates the dataset.

Weighted-averaging

3 **Weighted-averaging** takes the precision and recall by class and weights each of them with proportion of samples in each class.

$$\text{Precision} = \sum_{i=1}^{C} w_c \text{Precision}_c, \quad \text{Recall} = \sum_{i=1}^{C} w_c \text{Recall}_c, \quad \text{and} \quad \text{F1} = 2\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- *Good ...*

  Actually, it is similar to macro-averaging if $w_c = \frac{1}{N}$ for all $c \in \{1, 2, ..., C\}$. You can actually set the weight representation w_c for each class to solve the problem of macro-averaging.

- *Bad...*

  There is no rule as to how each w_c should be set. If you want to emphasize on the class with small samples, you can construct w_c to be inversely proportional to the number of samples in each class.

Summary: micro, macro, and weighted-averaging

Three common ways summarize Precision and Recall across different classes as one value:

1. **Macro-averaging** computes each precision_c and then average.

$$\text{Precision} = \frac{1}{C}\sum_{c=1}^{C}\frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c}, \quad \text{Recall} = \frac{1}{C}\sum_{c=1}^{C}\frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c}, \quad \text{and} \quad \text{F1} = 2\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

2. **Micro-averaging** takes a sum of all TP_c's and then compute precision/recall.

$$\text{Precision} = \frac{\sum_c \text{TP}_c}{\sum_c(\text{TP}_c + \text{FP}_c)}, \quad \text{Recall} = \frac{\sum_c \text{TP}_c}{\sum_c(\text{TP}_c + \text{FN}_c)}, \quad \text{and} \quad \text{F1} = \text{Precision} = \text{Recall}$$

   Note. Every FP for one class is an FN for another class. Thus, micro-average precision and micro-average will result in the same number.

3. **Weighted-averaging** takes the precision and recall by class and weights each of them with proportion of samples in each class.

$$\text{Precision} = \sum_{i=1}^{C} w_c \text{Precision}_c, \quad \text{Recall} = \sum_{i=1}^{C} w_c \text{Recall}_c, \quad \text{and} \quad \text{F1} = 2\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

References I

[1] *Accuracy, precision, and recall in multi-class classification*. https://www.evidentlyai.com/classification-metrics/multi-class-metrics. Accessed: 2025-12-11.

[2] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer Series in Statistics. New York, NY, USA: Springer New York Inc., 2001.

[3] Gareth James et al. *An Introduction to Statistical Learning: with Applications in R*. Springer, 2013. URL: https://faculty.marshall.usc.edu/gareth-james/ISL/.

[4] Jitkomut Songsiri. "Statistical Learning". In: in EE575 Random Processes for EE. Chulalongkorn University, 2022. Chap. 8. Introduction to Classification. URL: http://jitkomut.eng.chula.ac.th/ee575.html.